

PrepWise — Tam Proje Dokümantasyonu

Projeyi kısa cümlede tanımlarsak: PrepWise, ham (kırı) veri dosyalarını yükleyip yapay zeka yardımıyla analiz eden, eksik değer / aykırı değer / format sorunlarını otomatik tespit edip kullanıcının seçtiği yöntemlerle temizleyen ve temizlenmiş veriyi indirilebilir hale getiren bir **web uygulamasıdır**.

İçindekiler

- [Projenin Amacı ve Problemi](#)
- [Teknoloji Yığını \(Tech Stack\)](#)
- [Proje Klasör Yapısı](#)
- [Mimari — Büyük Resim](#)
- [Veritabanı Modelleri](#)
- [Backend — Katman Katman Anlatım](#)
 - 6.1 Kimlik Doğrulama (JWT)
 - 6.2 API Rotaları (Routes)
 - 6.3 Arka Plan Görevleri (Background Tasks)
 - 6.4 Veri Analiz Modülleri
 - 6.5 AI Asistan (Gemini)
- [Veri Temizleme Pipeline'ı — Adım Adım](#)
- [Frontend — Ekranlar ve Bileşenler](#)
- [Kullanıcı Akışı \(User Journey\)](#)
- [Güvenlik Önlemleri](#)
- [Docker ve Dağıtım \(Deployment\)](#)
- [Ortam Değişkenleri \(.env\)](#)
- [Veri Akış Diyagramı](#)
- [Önemli Tasarım Kararları](#)

1. Projenin Amacı ve Problemi

Problem

Makine öğrenmesi veya veri analizi yapan herkes şunu bilir: Ham veri nadiren temizdir. Gerçek dünya veri setlerinde:

- Eksik hücreler (NaN değerleri)
- Uç değerler / aykırı gözlemler (outlier)
- Yanlış veri tipleri (sayı metin olarak kayıtlı)
- Büyük/küçük harf tutarsızlıkları ("Erkek" vs "erkek")
- Fuzzy duplicate'ler ("İstanbul" ve "İstanbull" gibi yazım hataları)

Bu sorunlar model başarımını ciddi ölçüde düşürür ve elle düzeltmek saatler alır.

Çözüm

PrepWise, bu süreci otomatize eder:

- Kullanıcı CSV / XLSX / TXT dosyasını yükler
- Backend yapay zeka ile veriyi analiz eder
- Sistem her sütun için özelleştirilmiş yöntem önerileri sunar
- Kullanıcı uygulamak istediği yöntemleri seçer
- Sistem veriyi temizler, kalite raporu üretir
- Kullanıcı temiz veriyi indirir

2. Teknoloji Yığını (Tech Stack)

Backend

Teknoloji	Sürüm	Amaç
Python	3.x	Ana dil
FastAPI	0.129	REST API framework
SQLAlchemy	2.0	ORM (veritabanı etkileşimi)
SQLite	-	Geliştirme ortamı veritabanı
PostgreSQL	15	Üretim ortamı veritabanı
Pandas	2.3	Veri işleme
NumPy	2.4	Sayısal hesaplama
scikit-learn	1.8	ML algoritmaları (KNN, IsolationForest, DBSCAN, MICE)
PyJWT	2.12	JWT token yönetimi
bcrypt	5.0	Şifre hashleme
SlowAPI	0.1.9	Rate limiting
ReportLab	4.4	PDF rapor üretimi
Uvicorn	0.41	ASGI sunucu
requests	2.32	Gemini API çağrılar

Frontend

Teknoloji	Amaç
React	UI framework

Vite Build tool
Vanilla CSS Stillendime
Lucide React ikonlar

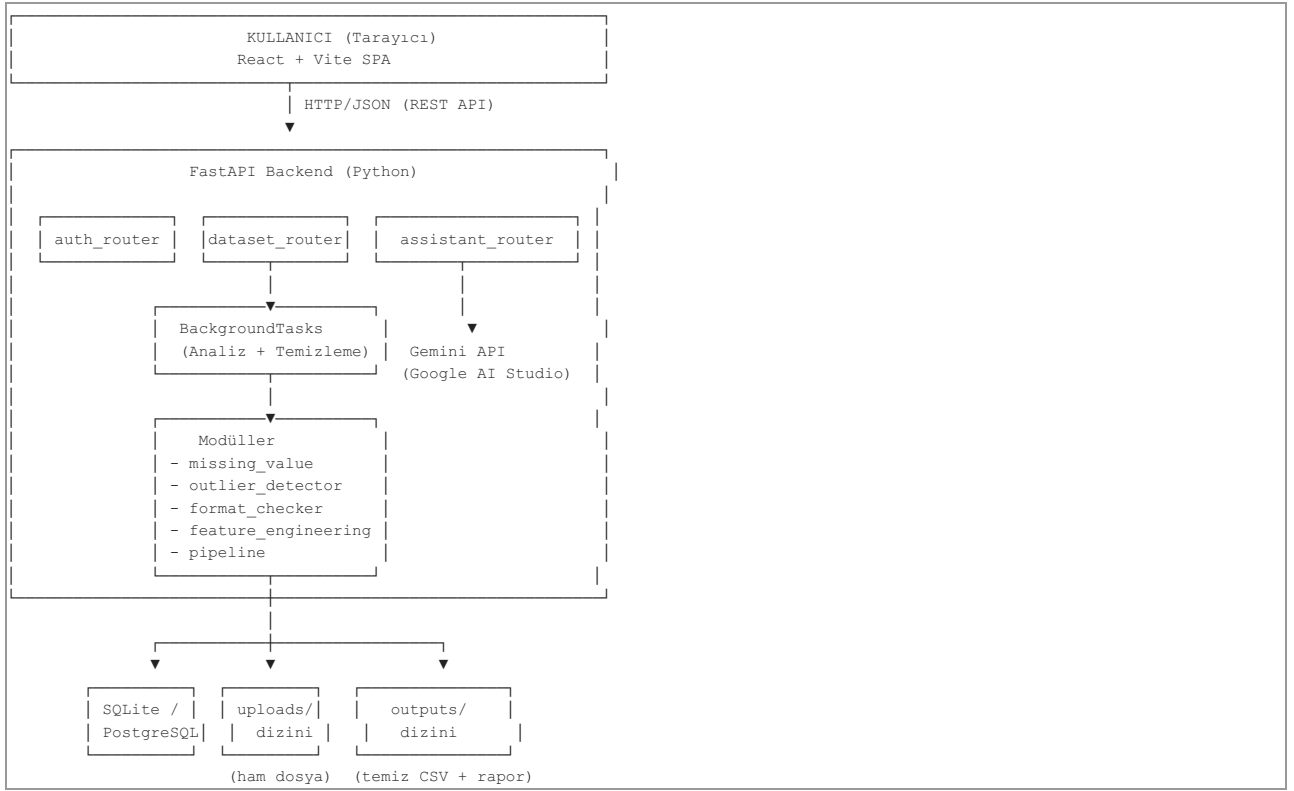
Altyapı

Teknoloji	Amaç
Docker	Container
Docker Compose	Çoklu servis orkestrasyonu
Nginx	(Frontend container'ında) reverse proxy
Google Gemini API	AI asistan chatbot

3. Proje Klasör Yapısı

```
ai-data-cleaner/
├── backend/                                ← Python / FastAPI backend
│   ├── main.py                            ← Uygulama giriş noktası
│   ├── auth.py                           ← JWT kimlik doğrulama
│   ├── database.py                       ← Veritabanı modelleri ve migrasyon
│   ├── assistant_gemini.py              ← Gemini AI asistan entegrasyonu
│   ├── environment.py                   ← .env yükleme yardımcısı
│   ├── requirements.txt                 ← Python paket listesi
│   └── Dockerfile                       ← Backend container tanımı
│
│   ├── core/                            ← Paylaşılan yardımcı modüller
│   │   ├── background_tasks.py          ← Analiz ve temizleme pipeline yöneticisi
│   │   ├── helpers.py                  ← Ortak yardımcı fonksiyonlar
│   │   ├── constants.py                ← Sabit değerler (dizin yolları vb.)
│   │   └── logging_config.py           ← Log yapılandırması
│   │
│   ├── modules/                         ← Veri analiz ve temizleme modülleri
│   │   ├── file_reader.py              ← CSV/XLSX/TXT okuyucu
│   │   ├── missing_value.py            ← Eksik değer analizi ve doldurma
│   │   ├── outlier_detector.py         ← Aykırı değer tespiti ve işleme
│   │   ├── format_checker.py           ← Format hatası tespiti ve düzeltme
│   │   ├── feature_engineering.py      ← Özellik mühendisliği
│   │   ├── recommendation.py          ← Tüm analiz modüllerini birleştirip öneri üretir
│   │   └── pipeline.py                 ← Temizleme pipeline orchestrator
│   │
│   ├── routers/                        ← FastAPI endpoint'leri
│   │   ├── auth_router.py              ← /register, /login, /me, şifre yönetimi
│   │   ├── dataset_router.py           ← /upload, /analyze, /apply, /download
│   │   ├── project_router.py           ← Proje CRUD işlemleri
│   │   ├── template_router.py          ← Temizleme şablonu CRUD
│   │   └── assistant_router.py         ← /assistant/chat endpointi
│   │
│   └── reporting/                      ← Kalite raporu üretimi (HTML/PDF)
│
├── frontend-new/                        ← React / Vite frontend
│   └── src/
│       ├── App.jsx                     ← Ana uygulama, routing, state yönetimi
│       ├── components/                 ← UI bileşenleri
│       │   ├── Hero.jsx                ← Açılış hero bölümü
│       │   ├── FileUpload.jsx          ← Dosya yükleme alanı
│       │   ├── AuthModal.jsx           ← Giriş/kayıt modal'ı
│       │   ├── UserDashboard.jsx       ← Kullanıcı paneli
│       │   ├── DatasetWorkspace.jsx    ← Dataset görüntüleme/stüdyo
│       │   ├── Chatbot.jsx             ← AI sohbet botu
│       │   ├── AccountSettings.jsx     ← Hesap ayarları
│       │   ├── FAQ.jsx                 ← Sıkça sorulan sorular
│       │   └── workspace/              ← Analiz ve temizleme arayüzü bileşenleri
│       │       ├── AnalysisResults.jsx  ← Öneri kartları ve seçim arayüzü
│       │       ├── ProfileView.jsx      ← Veri profil görünümü
│       │       └── ComparisonView.jsx   ← Önce/sonra karşılaştırması
│       └── services/
│           └── api.js                  ← Backend ile iletişim (fetch wrapper'ları)
│
├── uploads/                            ← Yüklenen ham dosyalar (runtime'da oluşur)
├── outputs/                             ← Temizlenmiş dosyalar ve raporlar
├── tests/                               ← Pytest test dosyaları
├── docker-compose.yml                  ← Docker Compose konfigürasyonu
├── .env                                ← Ortam değişkenleri (gizli, git'te yok)
└── .env.example                       ← Örnek ortam değişkenleri
```

4. Mimari — Büyük Resim



5. Veritabanı Modelleri

Proje SQLAlchemy ORM kullanır. Tüm modeller [database.py](file:///c:/Users/busra/Desktop/ai-data-cleaner/backend/database.py)'de tanımlıdır.

Tablolar ve İlişkileri

```
users
├── id (PK)
├── email (unique)
├── full_name
├── hashed_password
└── created_at
├── < projects (user_id FK)
│   ├── id, user_id, name, description, created_at
│   └── < datasets (user_id FK)
│       ├── id, user_id, project_id (-projects)
│       ├── filename (saklama adı, UUID bazlı)
│       ├── original_filename (original adı)
│       ├── format (csv/xlsx/txt)
│       ├── row_count, col_count
│       ├── file_path
│       ├── status → ready | analyzing | processing | cleaned | error
│       └── upload_time
│           ├── < cleaning_logs (dataset_id FK)
│           │   ├── module (missing/outlier/format/feature/duplicate)
│           │   ├── column_name, method, details
│           │   └── applied_at
│           └── < quality_reports (dataset_id FK)
│               ├── before_missing, after_missing
│               ├── outlier_count, format_errors
│               └── report_path (HTML dosyası yolu)
├── < cleaning_templates (user_id FK)
│   ├── name
│   └── selections_json (kaydedilen seçimler)
└── < password_reset_tokens (user_id FK)
    ├── token (UUID hex, 128 char)
    └── expires_at (1 saat geçerli)
```

Dataset Status Durum Makinesi

upload → ready
ready → analyzing (analiz tetiklendiğinde)
analyzing → ready (analiz bitince)
analyzing → error (analiz **başarısız**)
ready → processing (temizleme **başladığında**)
processing → cleaned (temizleme **başarılı**)
processing → error (temizleme **başarısız**)

Migrasyon Stratejisi

Proje, tam teşekküllü Alembic yerine `run_light_migrations()` adlı özel bir fonksiyon kullanır. Bu fonksiyon uygulama her başladığında çalışır ve eksik sütunları / foreign key kısıtlamalarını eker. Bu yaklaşım basit deployment senaryoları için pratiktir.

6. Backend — Katman Katman Anlatım

6.1 Kimlik Doğrulama (JWT)

[auth.py](file:///c:/Users/busra/Desktop/ai-data-cleaner/backend/auth.py) dosyası şunları içerir:

Şifre İşleme

```
# bcrypt ile hash'leme — 1-yönlü dönüşüm
get_password_hash(password) # kayıt sırasında
verify_password(plain, hashed) # giriş sırasında
```

Token Sistemi — Çift Token Mimarisi

Token Türü	Süre	Amaç
Access Token	30 dakika	API isteklerinde Authorization: Bearer
Refresh Token	7 gün	Access token yenileme

Access token süresi dolduğunda frontend otomatik olarak `/refresh` endpoint'ini çağırır ve yeni token alır. Her iki token da süresi dolarsa `auth:expired custom event` fırlatılır ve kullanıcı otomatik çıkış yapılır, login modal'ı açılır.

SECRET_KEY Yönetimi

Çevre değişkeni set edilmemişse, sistem `outputs/.jwt_secret` dosyasına kalıcı rastgele bir key kaydeder. Bu sayede sunucu yeniden başladığında mevcut oturumlar geçersiz olmaz.

get_current_user Bağımlılığı

FastAPI'nin dependency injection sistemi ile her korumalı endpoint `Depends(get_current_user)` alır. Bu fonksiyon Authorization header'ını parse eder, token'ı doğrular ve veritabanından kullanıcıyı döner.

6.2 API Rotaları (Routers)

Tüm rotalar `/api/v1/` prefix'i altındadır.

auth_router.py — Kimlik Yönetimi

Method	Endpoint	Açıklama	Rate Limit
POST	<code>/register</code>	Yeni kullanıcı kaydı	3/dakika
POST	<code>/login</code>	Giriş, access + refresh token döner	5/dakika
POST	<code>/refresh</code>	Refresh token ile yeni access token -	
GET	<code>/me</code>	Giriş yapmış kullanıcının bilgisi	-
GET	<code>/me/account</code>	Kapsamlı hesap özeti (istatistikler) -	
PATCH	<code>/me/profile</code>	İsim güncelleme	-
POST	<code>/me/password</code>	Şifre değiştirme	-
POST	<code>/forgot-password</code>	Şifre sıfırlama e-postası gönder	3/dakika
POST	<code>/reset-password</code>	Token ile yeni şifre belirle	-

dataset_router.py — Veri Seti İşlemleri

Method	Endpoint	Açıklama
POST	<code>/upload</code>	Ham dosya yükle (max 20 MB, CSV/XLSX/TXT)
GET	<code>/me/datasets</code>	Kullanıcının veri setleri + projeler
GET	<code>/analyze/{id}</code>	Analiz başlat (arka planda çalışır)
POST	<code>/apply/{id}</code>	Seçilen temizleme yöntemlerini uygula
GET	<code>/datasets/{id}/status</code>	Mevcut işlem durumu (polling için)
GET	<code>/datasets/{id}/workspace</code>	Profil + karşılaştırma verisi
GET	<code>/datasets/{id}/audit</code>	Tüm işlem geçmişi (denetim izi)
GET	<code>/datasets/{id}/audit-export</code>	Denetim izini CSV olarak indir
GET	<code>/download/{id}</code>	Temizlenmiş veriyi CSV olarak indir
GET	<code>/datasets/{id}/report</code>	Kalite raporunu HTML/PDF indir
DELETE	<code>/datasets/{id}</code>	Veri seti ve tüm ilişkili dosyaları sil
PUT	<code>/datasets/{id}/project</code>	Veri setini bir projeye ata

Dosya Yükleme Güvenliği:

```
# 1. Uzantı kontrolü
ext not in [".csv", ".txt", ".xlsx"] → 400

# 2. Boyut kontrolü (streaming, 1 MB'lik parçalar)
size > 20 * 1024 * 1024 → 400 + dosyayı sil

# 3. İçerik doğrulama
read_file(file_path) # okuma başarısız → 400 + dosyayı sil

# 4. UUID ile saklama (orijinal ad saklanır)
storage_name = f"{uuid.uuid4().hex}{ext}"
```

6.3 Arka Plan Görevleri (Background Tasks)

[background_tasks.py](file:///c:/Users/busra/Desktop/ai-data-cleaner/backend/core/background_tasks.py) — FastAPI'nin BackgroundTasks mekanizması ile çalışır. İstekler anında yanıt döner, ağır işlemler arka planda çalışır.

_run_analysis_async(dataset_id, user_id)

- Veri setini diskten oku
- get_basic_profile(df) — istatistiksel profil çıkar
- generate_recommendations(df) — 4 modülü çalıştır, öneri listesi üret
- Sonuçları outputs/analysis_{id}.json dosyasına kaydet
- Dataset status → "ready"

_apply_selections_to_dataset_async(dataset_id, user_id, selections)

Thread güvenliği: Her dataset_id için ayrı threading.Lock kullanılır. Aynı veri seti için çift temizleme girişimi engellenir.

İşlem adımları:

- Ham dosyayı oku
- run_pipeline(df, selections) — seçimleri sırayla uygula
- Sonucu önce .tmp geçici dosyasına kaydet
- Sağlık skoru hesapla (önce/sonra)
- Kalite raporu (HTML + PDF) üret
- Atomik dosya değişimi:** eski cleaned → .bak, yeni .tmp → cleaned
- DB'ye CleaningLog ve QualityReport kayıtlarını yaz
- db.commit() başarılı → .bak dosyasını sil
- db.commit() başarısız → .bak'ı geri yükle (rollback)
- Eski analiz cache'ini sil (temizleme sonrası yeni analiz yapıldığında taze sonuç görmek için)
- Dataset status → "cleaned"

Hata durumunda: Tüm geçici dosyalar temizlenir, status → "error"

6.4 Veri Analiz Modülleri

file_reader.py

CSV, XLSX ve TXT formatlarını okur. UTF-8 ve Latin-1 encoding denemesi yapılır. Döndürülen değer: (DataFrame, meta_dict) — meta_dict'te format, row_count, col_count bulunur.

missing_value.py — Eksik Değer Modülü

analyze_missing(df) — Her sütun için analiz:

- Eksik hücre sayısı ve yüzdesi
- Sütun tipi (sayısal / kategorik)
- Tipe uygun yöntem önerileri

Sayısal sütunlar için önerilen yöntemler:

Yöntem	Teknik	Ne zaman kullanılır
MICE	IterativeImputer + ExtraTrees	En yüksek doğruluk gerektiğinde
KNN (k=5)	KNNImputer	Yüksek doğruluk, orta hız
Mean	SimpleImputer(strategy="mean")	Hızlı, basit
Median	SimpleImputer(strategy="median")	Aykırı değerlere dayanıklı
Drop	dropna(subset=[col])	Veri kaybı kabul edilebilirse

MICE (Multiple Imputation by Chained Equations) nedir?

Tüm sayısal sütunları girdi olarak alan bir Makine Öğrenmesi modeli (ExtraTreesRegressor) kurar. Eksik olan değer, diğer sütunlardaki desenleri öğrenerek tahmin edilir. Kurumsal veri bilimi projelerinde standart yöntemdir.

Kategorik sütunlar için:

Yöntem	Açıklama
Mode	En sık tekrar eden değerle doldurma
Constant	"Bilinmiyor" sabit etiketi
Drop	Satır sil

outlier_detector.py — Aykırı Değer Modülü

analyze_outliers(df) — 4 farklı algoritma ile analiz:

1. IQR (Interquartile Range)

```
Alt sınır = Q1 - 1.5 × IQR
Üst sınır = Q3 + 1.5 × IQR
```

Sütun bazında çalışır, deterministik sonuç verir.

2. Isolation Forest

Ağaç tabanlı ensemble yöntemi. Anomali noktaları "izole edilmesi kolay" noktalar olarak tanımlar. `contamination="auto"` ile gerçek anomali eşiği kullanılır (`decision_function < 0`).

3. Local Outlier Factor (LOF)

Bir noktanın yoğunluğunu komşularının yoğunluğuyla karşılaştırır. `negative_outlier_factor_ < -1.5` eşiği.

4. DBSCAN (Density-Based Spatial Clustering)

Yoğunluk bazlı kümeleme algoritması. -1 etiketli noktalar "gürültü" (noise) = aykırı değer.

Uygulama yöntemleri:

ID	Ad	Teknik
keep	Olduğu gibi bırak	Hiçbir şey yapma, sadece raporla
cap	Sınırla (Winsorize)	IQR sınırlarına clip et
percentile_cap	Oransal kırpma	%5-%95 aralığına clip et
drop_outliers	Aykırı satırları sil	IQR dışındakileri kaldır
median_replace	Medyan ile değiştir	Aykırı değerleri medyanla değiştir
dbscan_drop	DBSCAN ile sil	DBSCAN noise satırlarını kaldır
iso_forest_drop	Isolation Forest ile sil	Anomali olarak işaretlenenler kaldır

format_checker.py — Format Hata Modülü

5 farklı kontrol yapar:

Kontrol 1: Sayısal-olması-gereken metin sütunları

Hücrelerin %80'i sayıya dönüşebiliyorsa → `numeric_as_string` hatası.

Çözüm: `pd.to_numeric(col, errors='coerce')`

Kontrol 2: Tarih formatı tespiti

Regex ile 4 farklı tarih deseni aranır (YYYY-MM-DD, DD/MM/YYYY, DD.MM.YYYY, YYYY/MM/DD).

Eşleşme oranı > %50 ise → `date_as_string` hatası.

Çözüm: `pd.to_datetime(col, errors='coerce', dayfirst=True)`

Kontrol 3: Baştaki/sondaki boşluklar

`str.strip() != str` kontrolü — kaç hücrede boşluk var?

Çözüm: `str.strip()`

Kontrol 4: Büyük/küçük harf tutarsızlığı

Benzersiz değer sayısı 20'den az ise: lowercase versiyonları ile karşılaştır.

"Erkek" + "erkek" + "ERKEK" → 3 farklı değer, ama aslında 1.

Çözüm: `str.lower()`

Kontrol 5: Fuzzy (Bulanık) Benzerlik — Yazım Hataları

`difflib.SequenceMatcher` ile tüm benzersiz değer çiftleri karşılaştırılır.

Benzerlik oranı $0.85 \leq \text{ratio} < 1.0$ ise → `fuzzy_duplicates` hatası.

Çözüm: Daha sık geçen değere birleştir (semantik merge).

feature_engineering.py — Özellik Mühendisliği

Makine öğrenmesi modelleri için veri hazırlama:

Tarih sütunları için:

- `extract_date_features`: Yıl, ay, gün, haftanın günü sütunlarını türet

Kategorik sütunlar için:

- `one_hot_encode`: ≤15 benzersiz değer → ikili (0/1) sütunlar
- `label_encode`: >15 benzersiz değer → sayısal ID'ler

Sayısal sütunlar için:

- `log_transform`: Skewness > 1.0 ise logaritmik dönüşüm
- `standard_scale`: Z-score standartlaştırma ($\mu=0, \sigma=1$)
- `minmax_scale`: 0-1 aralığına sıkıştırma

recommendation.py — Öneri Motoru

Tüm modülleri `generate_recommendations(df)` altında birleştirir:

```
missing_analysis = analyze_missing(df)
outlier_analysis  = analyze_outliers(df)
format_analysis  = analyze_format(df)
feature_analysis  = analyze_features(df)
```

Her sorun için bir kart üretir:

```
{
  "id": "missing_yas",
  "category": "missing",
  "column": "yas",
  "summary": "yas: 23 eksik deęer (%11.5)",
  "severity": "medium",    ← low | medium | high
  "options": [...]        ← kullanıcıya sunulacak yöntemler
}
```

Severity (önem derecesi) kuralları:

- Eksik deęer > %30 → high
- Eksik deęer > %10 → medium
- Dięer → low

pipeline.py — Temizleme Orkestratörü

run_pipeline(df, selections) kullanıcının seçtięi yöntemleri sırayla uygular:

```
for selection in selections:
    category = selection["category"]    # missing / outlier / format / feature / duplicate
    column   = selection["column"]
    method   = selection["method"]

    if category == "missing":
        df, detail = apply_missing(df, column, method)
    elif category == "outlier":
        df, detail, count = apply_outlier(df, column, method)
    # ...
```

Sıra önemlidir: önce eksik deęer doldurma, sonra aykırı deęer işleme önerilir.

6.5 AI Asistan (Gemini)

[assistant_gemini.py](file:///c:/Users/bursa/Desktop/ai-data-cleaner/backend/assistant_gemini.py) — Google Gemini API entegrasyonu.

Model: gemini-2.5-flash-lite (çevre deęişkeniyle deęiştirilebilir)

Sistem Promptu: Asistan PrepWise'a özel kurallara sahiptir:

- Markdown yıldız işareti kullanmaz (arayüzde çirkin görünür)
- Düz metin + tire/rakam liste kullanır
- Sadece PrepWise özellikleri hakkında bilgi verir

Mesaj Yönetimi (token tasarrufu için):

- Son 5 mesaj gönderilir (geçmişin şımesini önler)
- İlk mesaj her zaman "user" rolünde olmalı (Gemini kuralı)
- Roller ardışık olmalı — aynı rol arka arkaya gelemez

Endpoint: POST /api/v1/assistant/chat

```
{
  "messages": [
    {"role": "user", "text": "Eksik deęer nedir?"},
    {"role": "model", "text": "Eksik deęer..."},
    {"role": "user", "text": "MICE yöntemini açıklar mısın?"}
  ]
}
```

7. Veri Temizleme Pipeline'ı — Adım Adım

```
1. YÜKLEME
  Kullanıcı → /upload
  └─ Dosya doğrulama (uzantı, boyut, içerik)
  └─ UUID ile diske kaydet (uploads/ klasörü)
  └─ Dataset kaydı oluştur (status="ready")

2. ANALİZ
  Kullanıcı → /analyze/{id}
  └─ Status "analyzing" yap
  └─ BackgroundTask başlat (_run_analysis_async)
  │   └─ missing_value analizi
  │   └─ outlier analizi (IQR + IsolationForest + LOF + DBSCAN)
  │   └─ format analizi (5 kontrol)
  │   └─ feature engineering analizi
  │   └─ Duplicate satır sayısı
  └─ Sonuçları outputs/analysis_{id}.json'a kaydet
  └─ Status "ready" yap

3. POLLİNG
  Frontend → /datasets/{id}/status
  Her 1.5 saniyede sorgular, status="ready" olunca durur

4. ÖNERİLERİ GÖRÜNTÜLE
  Frontend → /analyze/{id} (tekrar, önbellekten döner)
  Her sorun için kart gösterilir, kullanıcı yöntem seçer

5. TEMİZLEME UYGULAMA
  Kullanıcı → /apply/{id}
  └─ Status "processing" yap
  └─ BackgroundTask başlat (_apply_selections_to_dataset_async)
  │   └─ run_pipeline(df, selections) – sırayla uygula
  │   └─ Sağlık skoru hesapla (önce/sonra)
  │   └─ HTML + PDF rapor üret
  │   └─ Atomik dosya yazma (tmp → final)
  │   └─ DB'ye CleaningLog + QualityReport kaydet
  │   └─ Analiz cache'ini sil (yeni analiz için)
  └─ Status "cleaned" yap

6. İNDİRME
  Kullanıcı → /download/{id}
  outputs/cleaned_{filename}.csv döndürülür
```

8. Frontend — Ekranlar ve Bileşenler

Navigasyon (View State Makinesi)

Proje React Router kullanmaz. Bunun yerine `currentView` state değişkeni ile ekranlar render edilir.

View	Bileşen	Açıklama
home	Hero + FileUpload + FAQ + AnalysisCards	Açılış sayfası
panel	UserDashboard	Yüklenen veri setleri listesi
profile	DatasetWorkspace (ProfileView)	Veri seti profili
studio	DatasetWorkspace (AnalysisResults)	Analiz + temizleme stüdyosu
comparison	DatasetWorkspace (ComparisonView)	Önce/sonra karşılaştırması
account	AccountSettings	Profil ve şifre ayarları

Önemli Bileşenler

AuthModal.jsx

Tek modal içinde iki form: Giriş ve Kayıt.

- E-posta / şifre doğrulama
- Token'ları localStorage'a kaydeder
- Şifre sıfırlama akışı (e-posta gönder → kod al → yeni şifre)

FileUpload.jsx

- Drag-and-drop veya tıkla seç
- Proje seçimi (veri seti bir projeye eklenebilir)
- Yeni proje oluşturma
- Giriş yapılmamışsa auth modal'ı açar

AnalysisResults.jsx

En kritik bileşen:

- Her sorun için kart render edilir
- Kullanıcı her kart için bir yöntem seçer (radio button)
- Şablon kaydetme ve yükleme
- "Temizle" butonu selections listesini API'ye gönderir

DatasetWorkspace.jsx

Üç tab:

1. **Profil** — Sütun istatistikleri, veri tipleri, eksik değer oranları
2. **Stüdyo** — Analiz ve temizleme arayüzü
3. **Karşılaştırma** — Orijinal vs temizlenmiş veri yan yana

Chatbot.jsx

Floating chat butonu → sohbet penceresi.
Mesaj geçmişi tutulur, Gemini API'ye iletilir

UserDashboard.jsx

- Proje bazlı filtreleme
- Dataset kartları (status, satır/sütun sayısı, temizlenmiş mi?)
- Dataset silme

9. Kullanıcı Akışı (User Journey)



10. Güvenlik Önlemleri

Rate Limiting (Hız Sınırlama)

```
@limiter.limit("3/minute") # Kayıt - brute force önleme
@limiter.limit("5/minute") # Giriş
@limiter.limit("3/minute") # Şifre sıfırlama
```

IP bazlı, in-memory storage. Üretimde Redis önerilir.

Veri Yalıtımı

Her endpoint `user_id` kontrolü yapar. Kullanıcı A, kullanıcı B'nin veri setine erişemez:

```
dataset = db.query(Dataset).filter(
    Dataset.id == dataset_id,
    Dataset.user_id == user.id ← sahiplik kontrolü
).first()
```

Dosya Güvenliği

- Dosya uzantısı whitelist ile kontrol
- UUID bazlı saklama adı (orijinal adla erişim engellenir)
- 20 MB limit (streaming ile kontrol)
- Upload ve output izinleri birbirinden ayrı

Şifre Güvenliği

- Minimum 8 karakter (Pydantic field validator)
- bcrypt hashleme (yüksek cost faktörü)
- Şifre sıfırlama tokenları 1 saatte expire

JWT Güvenliği

- type claim kontrolü (access vs refresh)
- Şifre değiştirilince tüm reset tokenları silinir
- Yeni şifre eskiyle aynı olamaz

CORS

Sadece izin verilen origin'ler `CORS_ORIGINS` env değişkeninden yüklenir.

11. Docker ve Dağıtım (Deployment)

[docker-compose.yml](file:///c:/Users/busra/Desktop/ai-data-cleaner/docker-compose.yml) 3 servis tanımlar.

db — PostgreSQL

```
image: postgres:15-alpine
healthcheck: pg_isready ...
```

Backend başlamadan önce sağlıklı olması beklenir.

backend — FastAPI

```
build: backend/Dockerfile
ports: "8000:8000"
depends_on: db (service_healthy)
volumes:
  - ./uploads:/app/uploads
  - ./outputs:/app/outputs
```

Uploads ve outputs dizinleri host'a volume olarak bağlanır → container yeniden başlasa bile veriler kaybolmaz.

frontend — React (Nginx)

```
build: frontend-new/
ports: "80:80"
depends_on: backend (service_healthy)
```

Vite ile statik dosyaya derlenir, Nginx ile servis edilir.

Başlatma Sırası

PostgreSQL hazır → **Backend başlar** → Backend hazır → Frontend **başlar**

Yerel Geliştirme

Backend: `uvicorn backend.main:app --reload --port 8000`
Frontend: `npm run dev` (port 5173, CORS için izinli)

12. Ortam Değişkenleri (.env)

```
# Veritabanı
DATABASE_URL=postgresql://postgres:sifre@localhost:5432/cleaner_db
# SQLite geliştirme için: otomatik (boş bırakılabilir)

# JWT
SECRET_KEY=guclu-rastgele-anahtar # boş bırakılırsa auto-generated

# JWT süreleri (dakika)
ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_MINUTES=10080 # 7 gün

# E-posta (şifre sıfırlama için)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=ornek@gmail.com
SMTP_PASSWORD=uygulama-sifresi

# AI Asistan
GEMINI_API_KEY=Aiza...
GEMINI_MODEL=gemini-2.5-flash-lite

# CORS
CORS_ORIGINS=http://localhost:5173,http://localhost:80

# PostgreSQL (Docker Compose için)
POSTGRES_USER=postgres
POSTGRES_PASSWORD=guclu-sifre
POSTGRES_DB=cleaner_db
```

13. Veri Akış Diyagramı

```
sequenceDiagram
    actor U as Kullanıcı
    participant F as Frontend (React)
    participant B as Backend (FastAPI)
    participant BG as BackgroundTask
    participant DB as Veritabanı
    participant FS as Dosya Sistemi

    U->>F: Dosya seçer
    F->>B: POST /upload (multipart)
    B->>FS: UUID adıyla kaydet (uploads/)
    B->>DB: Dataset kaydı oluştur (status=ready)
    B-->>F: {dataset_id, meta}

    F->>B: GET /analyze/{id}
    B->>DB: status → analyzing
    B->>BG: _run_analysis_async başlat
    B-->>F: {status: "analyzing"}

    loop Polling (1.5 sn)
        F->>B: GET /datasets/{id}/status
        B-->>F: {status: "ready"}
    end

    F->>B: GET /analyze/{id} (cache'den)
    B-->>F: {recommendations: [...]}

    U->>F: Yöntemleri seçer, "Temizle" tıklar
    F->>B: POST /apply/{id} {selections: [...]}
    B->>DB: status → processing
    B->>BG: _apply_selections başlat
    B-->>F: {status: "processing"}

    BG->>FS: Ham dosyayı oku
    BG->>BG: run_pipeline() - temizle
    BG->>FS: cleaned_{file}.csv yaz (atomik)
    BG->>BG: HTML + PDF rapor üret
    BG->>DB: CleaningLog + QualityReport kaydet
    BG->>DB: status → cleaned

    loop Polling (2 sn)
        F->>B: GET /datasets/{id}/status
        B-->>F: {status: "cleaned"}
    end

    U->>F: "İndir" tıklar
    F->>B: GET /download/{id}
    B->>FS: outputs/cleaned_{file}.csv
    B-->>U: CSV dosyası
```

14. Önemli Tasarım Kararları

Neden React Router yok?

Proje, `currentView` state değişkeni ile ekranlar arasında geçiş yapar. Bu, URL bazlı routing olmadığı anlamına gelir (sayfa yenilenince ana sayfaya döner). Basit bir SPA için bu yaklaşım yeterlidir ve bağımlılığı azaltır.

Neden BackgroundTasks, Celery değil?

FastAPI'nin yerleşik `BackgroundTasks` mekanizması kullanılır. Celery/Redis gibi harici bir iş kuyruğu altyapısına gerek kalmaz. Dezavantajı: çok sayıda eş zamanlı kullanıcıda thread havuzu tükenilebilir. Ölçekleme gerekirse Celery + Redis geçişi planlanabilir.

Neden Alembic yok?

`run_light_migrations()` fonksiyonu startup'ta çalışarak sütun/FK ekler. Bu yaklaşım küçük projeler için pratiktir, schema değişiklikleri sık olmadığına yeterlidir.

Neden SQLite (geliştirme) / PostgreSQL (üretim)?

SQLite kurulum gerektirmez, geliştirme sürecini hızlandırır. PostgreSQL üretimde concurrent yazma sorunlarını çözer, JSON operatörleri ve gelişmiş index desteği sağlar.

Atomik Dosya Yazma

```
eski cleaned → .bak (yedek)
yeni .tmp → cleaned (commit)
db.commit() OK → .bak sil
db.commit() FAIL → .bak'ı geri yükle
```

Güç kesintisi veya hata durumunda veri kaybı/bozulması önlenir.

Dataset başına Thread Lock

```
_dataset_locks: dict[int, threading.Lock] = {}
```

Aynı veri setine çift temizleme isteği gelirse ikinci istek birincisinin bitmesini bekler. Race condition önlenir.

Analiz Önbellesi

Analiz sonuçları JSON dosyasına kaydedilir. Aynı dosya için tekrar analiz istenirse diskten okunur. Temizleme yapıldığında bu cache silinir — kullanıcı taze (temizlenmiş veriye ait) analiz sonuçları görür.

Bu dokümantasyon projenin tüm katmanlarını kapsamaktadır. Her dosya için dosya bağlantıları yukarıda verilmiştir.